Software Requirements Specification (SRS)

MedLLM Interface: AI Platform for Empathy and Communication Training in Healthcare

Document Information

Field	Details
Version	0.1
Date	October 12, 2025
Team	Christian Canlas, Asher Haroon, Michael Fedotov, Hammad Ur Rehman, Tung Ho, Aaryan Kandwal, Samir Matani
Supervisor	Dr. Allison Lahnala
Instructor	Dr. Mehdi Moradi
Course	COMPSCI 4ZP6A – Capstone Project
Project GitHub	https://github.com/AsherHaroon/capstone

Version History

Version	Author(s)	Description	Date
0.1	Team 16	Initial draft of Software Requirements Specification (based on supervisor feedback and deliverables v0.3).	Oct 12, 2025

Table of Contents

1. Purpose of the Project
    - 1.1 Background
    - 1.2 Problem Statement
    - 1.3 Project Objectives
    - 1.4 Proposed Solution
2. Stakeholders
    - 2.1 Primary Stakeholders
    - 2.2 Secondary Stakeholders
    - 2.3 User Participation
3. Project Constraints
    - 3.1 Partner or Collaborative Application Constraints
    - 3.2 Off-the-Shelf Software
    - 3.3 Environment Constraints
    - 3.4 Schedule Constraints
    - 3.5 Budget Constraints
    - 3.6 Enterprise and Compliance Constraints
4. Data and Metrics
    - 4.1 Data Overview
    - 4.2 Data Schema
    - 4.3 Metrics and Evaluation
5. Scope
    - 5.1 Scope of the Work
    - 5.2 Scope of the Product
6. Functional Requirements
    - 6.1 Priority 0 — Core MVP
    - 6.2 Priority 1 — Enhanced Functionality
    - 6.3 Priority 2 — Future / Multimodal Expansion
7. Non-Functional Requirements
    - 7.1 Data Requirements
    - 7.2 Usability and Humanity Requirements
    - 7.3 Performance Requirements
    - 7.4 Operational and Environmental Requirements
    - 7.5 Maintainability and Support Requirements
    - 7.6 Security and Privacy Requirements
    - 7.7 Compliance Requirements
    - 7.8 User Documentation and Training
8. Risks and Open Issues
    - 8.1 Technical and Ethical Risks
    - 8.2 Open Problems and Mitigation
9. Glossary
10. Appendix A – Dialogue System Architecture Diagram

1. Purpose of the Project

1.1 Background

Effective communication and empathy are essential competencies in clinical practice, yet existing training methods (e.g., standardized patient programs) are resource-intensive, limited in scalability, and subject to human variability. With advancements in large language models (LLMs), it is now possible to simulate realistic patient interactions that respond contextually and emotionally to trainees' communication.

1.2 Problem Statement

Medical trainees lack accessible, scalable tools for practicing empathetic dialogue and receiving actionable feedback. Current simulation and standardized patient programs require human actors and significant faculty supervision, making continuous, low-cost practice unattainable.

1.3 Project Objectives

- Design a **web-based conversational platform** for empathy and communication training.
- Implement a **Dialogue System Architecture** using NLU, Dialogue Management, and NLG.
- Provide **automated feedback** aligned with the SPIKES protocol and Appraisal Framework for Clinical Empathy.
- Enable **secure data storage** and **research API access** for ongoing studies.
- Ensure extensibility for future multimodal enhancements (audio, video, emotion recognition).

1.4 Proposed Solution

The **MedLLM Interface** will allow medical trainees to select a virtual patient case, engage in an AI-simulated conversation, and receive structured feedback on empathy, communication quality, and SPIKES-stage coverage.

The platform will consist of:

- **Learner-facing UI:** Chat-based interface for conversation and feedback visualization.
- **Admin/Research UI:** Dashboard for managing users, cases, and analytics.
- **Dialogue System Backend:** Modular NLU–DM–NLG pipeline with a REST API.
- **Database & Case Repository:** Stores synthetic patient personas and session data.

2. Stakeholders

2.1 Primary Stakeholders

Primary stakeholders include medical trainees who use the learner-facing interface to practice empathetic communication with virtual patients and receive automated feedback. Clinical communication instructors use the administrative dashboard to assign cases, monitor progress, and review transcripts. Dr. Allison Lahnala supervises the project's research direction and integration of the dialogue system architecture, ensuring alignment with educational and technical goals. Dr. Mehdi Moradi and the teaching assistants oversee

progress, evaluate deliverables, and assess the proof of concept. The development team is responsible for the design, implementation, testing, and documentation of the system.

2.2 Secondary Stakeholders

Secondary stakeholders include healthcare simulation researchers who may adapt the MedLLM Interface for academic use and future developers who may extend its modular architecture for multimodal or multilingual support. Institutional IT administrators may assist with deployment and authentication through systems such as CAS or Brightspace.

2.3 User Participation

Medical trainees will engage with virtual patients through the web interface to complete communication exercises. Instructors and researchers will review anonymized transcripts and feedback using the dashboard and research API, while the supervisor and instructor provide guidance and validation throughout development.

3. Project Constraints

3.1 Partner or Collaborative Application Constraints

- Integration must comply with McMaster's research ethics and PIPEDA privacy standards.
- No personally identifiable information (PHI) may be stored or processed.

3.2 Off-the-Shelf Software

FastAPI: Backend service and REST API.

PostgreSQL: Database for user sessions and case data.

Google Gemini API / OpenAI GPT API: LLM for dialogue generation.

React: Frontend interface.

Docker/OpenShift: Containerization and deployment.

3.3 Environment Constraints

- Hosted on McMaster or cloud infrastructure (CAS private cloud or GCP).
- Frontend accessible on major browsers (Chrome, Edge, Firefox).
- GPU access is optional for model inference.

3.4 Schedule Constraints

Team/Project Approval: Sept 19, 2025

SRS & Project Plan Submission: Oct 12, 2025

Proof of Concept Demo: Nov 21, 2025

Design + V&V Document: Jan 23, 2026

Final Submission: Apr 4, 2026

3.5 Budget Constraints

- All tools must have free academic or open-source tiers.
- Total compute cost must remain under $40 per month during development.
- This estimate covers limited paid API usage (e.g., OpenAI or Gemini) and occasional cloud compute costs, while core tools and infrastructure run on free academic or open-source tiers.

3.6 Enterprise and Compliance Constraints

- Must comply with PIPEDA (Canada) and ethical AI standards.
- Source code licensed under MIT License.
- All LLM usage must avoid storing user data outside Canada where feasible.

4. Data and Metrics

4.1 Data Overview

MedLLM uses synthetic, anonymized data to maintain privacy and ethical compliance. No personal health information (PHI) is stored, and all patient cases are system-generated under the SPIKES framework. Dialogue and feedback data are stored in PostgreSQL with encrypted backups. Admins can export anonymized datasets via read-only API access.

Key data types include:

- Synthetic patient cases aligned with the SPIKES framework.
- Dialogue-state data tracked by the **Dialogue State Management System** (context, tone, learner patterns).
- Feedback summaries assessing empathy and communication quality.
- Metadata such as anonymized user IDs and timestamps.

4.2 Data Schema

Each session is saved in JSON format with the case ID, dialogue history, and feedback metrics, linked to non-identifiable user IDs. The Dialogue State Management System tracks patient states and learner behaviours throughout the session.

```
{
 "session_id": "uuid",
 "timestamp_start_utc": "2025-10-10T14:23:05.123Z",
 "conversation_history": [
        {
        "turn_number": 1, "speaker": "learner", "text": "Hello, I'm Dr. Smith...",
        "timestamp_utc": "2025-10-10T14:23:05.123Z", "analysis": {...}
        }
 ],
 "current_stage": "information_gathering",
 "patient_state": {
        "emotional_state": "anxious", "discussed_topics": ["diagnosis", "family"],
        "primary_concern": "treatment_options", "analysis": {...}
 },
 "learner_patterns": {
  "empathy_attempts": 3, "open_questions": 5, "interruptions": 1
```

```
  }
}
```

4.3 Metrics and Evaluation

MedLLM tracks both technical and educational metrics to evaluate dialogue quality, empathy accuracy, and overall performance. Fairness is assessed through Bias Probe Consistency, ensuring empathy scores stay consistent across demographically varied cases. Feedback Reliability measures how closely automated empathy scores align with instructor-assigned scores, validating that AI-generated feedback accurately reflects human judgment. Performance and feedback metrics are validated through pilot testing and iterative refinement.

Category	Metric	Description	Source
Empathy & Communication	Empathy Score (0–1)	Weighted empathy index from detected phrases, tone, and SPIKES coverage.	Feedback Analyzer
	Open-Question Ratio	% of open-ended questions.	NLU Analysis
	Reassurance Count	Count of supportive or validating phrases.	Text Analysis
	SPIKES Stage Coverage	% of SPIKES stages covered.	Dialogue Manager
Technical Performance	Dialogue Latency	Input–response time ≤ 2 s.	System Logs
	Uptime	Platform availability ≥ 99%.	Server Monitoring
	Session Capacity	Stable concurrent sessions ≥ 10.	Load Testing
Data Quality & Fairness	Bias Probe Consistency	Empathy score variation < 10% across demographics.	Bias Probe Mode
	Feedback Reliability	AI–instructor score correlation ($r \geq 0.8$).	Validation Studies

5. Scope

5.1 Scope of the Work

Team 16 will design, implement, test, and document the MedLLM Interface, encompassing the frontend (React UI), backend (FastAPI), dialogue pipeline (NLU–DM–NLG), and database layer (PostgreSQL) to ensure full system integration and functionality.

5.2 Scope of the Product

The system will provide a secure login portal that allows users to select virtual patient cases and engage in real-time, text-based conversations with the AI. It will automatically record transcripts, generate feedback, and enable instructors to review analytics through an integrated admin dashboard. The platform will also be extensible, supporting future enhancements such as audio, video, and emotion recognition modules.

6. Functional Requirements

6.1 Priority 0 — Core MVP

ID	Requirement	Description
FR-1	Login System	Users shall authenticate with role-based access (Trainee, Admin).
FR-2	Case Selection	Users shall choose from preloaded patient scenarios.
FR-3	Chat Interface	Users shall converse with AI patients via text input/output.
FR-4	Dialogue System Core	System shall implement a 3-component pipeline: NLU → Dialogue Manager → NLG.
FR-5	Session Logging	The system shall log all conversation turns with timestamps and case IDs.
FR-6	Feedback Summary	The system shall provide textual feedback summarizing empathy performance.
FR-7	Admin Logs	Admins shall view user transcripts and feedback summaries.
FR-8	Research API	Read-only endpoints for session retrieval and export.
FR-9	HTTPS + Security	All connections shall be HTTPS with no PHI or PII storage.

6.2 Priority 1 — Enhanced Functionality

ID	Requirement	Description
FR-10	Audio Processing	The system shall support speech input/output (ASR + TTS).
FR-11	Model Gateway	The system shall allow switching between Gemini, GPT, or local models.
FR-12	Feedback Visualization	The system shall highlight empathy opportunities and SPIKES coverage.
FR-13	Analytics Dashboard	Admin dashboard shall display trends and empathy score graphs.
FR-14	Case Authoring	Admins shall create/edit patient cases.
FR-15	Bias Probe Mode	System shall replay same case with varied demographics for fairness testing.

6.3 Priority 2 — Future / Multimodal Expansion

ID	Requirement	Description
FR-16	Virtual Patient Avatar	Display animated avatar with emotional expressions.
FR-17	Webcam Emotion Recognition	Analyze trainee facial expressions for emotion matching.
FR-18	Empathy Event Tagging	Automatically label empathy events (EO ↔ Response ↔ SPIKES).
FR-19	Multilingual Support	Support dialogues in English and French.
FR-20	LMS Integration	Integrate with institutional SSO or Brightspace.

7. Non-Functional Requirements

7.1 Data Requirements

- Database must store: session transcripts, feedback, user metadata, and case info.
- All data is stored anonymized, using unique session IDs.

7.2 Usability and Humanity Requirements

- The learner interface must display clear dialogue flow and immediate feedback.
- Interface design must prioritize accessibility (WCAG 2.1 compliant).

7.3 Performance Requirements

- Dialogue/Response latency (input → response) ≤ 2 seconds.
- Support 10 concurrent sessions on a standard server.
- System uptime ≥ 99% during evaluation.

7.4 Operational and Environmental Requirements

- Deployed in Docker containers for portability.
- Accessible via desktop and mobile browsers.
- Requires Internet connectivity for inference.

7.5 Maintainability and Support Requirements

- All modules documented via OpenAPI and code docstrings.
- Dialogue System must be modular (components replaceable).
- GitHub used for version control and issue tracking.

7.6 Security and Privacy Requirements

- All user data encrypted in transit (TLS 1.2+).
- No PHI and no PII are stored. All sessions are anonymized.
- Admin access restricted by role-based authentication.
- Audit logging for all data exports.

7.7 Compliance Requirements

- PIPEDA-compliant handling of synthetic conversations.
- LLM usage adheres to responsible AI research guidelines.

- MIT License to govern open-source release.

7.8 User Documentation and Training

- README for setup and operation.
- API documentation (Swagger / OpenAPI).
- Tutorial video for learners and admin users.

8. Risks and Open Issues

8.1 Technical and Ethical Risks

Risk	Description	Mitigation
LLM Bias	The model may produce biased or insensitive responses.	Use diverse synthetic cases; add manual review filters.
Data Privacy	Risk of storing identifiable dialogue.	Use synthetic and anonymized cases only.
API Cost	Cloud LLM usage may exceed the budget.	Implement model caching; use free-tier credits.
Latency	LLM inference delay may affect realism.	Optimize batching and response truncation.
Research Ethics	AI-generated feedback might be misinterpreted.	Include disclaimers and a human supervision option.

8.2 Open Problems and Mitigation

- Integration of Empathy Scoring Models in Research.
- Ongoing evaluation for bias and interpretability.
- Future testing with real clinical trainees is pending REB approval.

9. Glossary

ASR (Automatic Speech Recognition): Converts spoken input from the trainee into text for processing.

Dialogue Management (DM): Tracks context and state of the conversation.

LLM: Large Language Model used to simulate patient responses.

MIT License: Permissive open-source license allowing reuse and modification with attribution.

NLG: Natural Language Generation — produces system output text.

NLU: Natural Language Understanding — processes user input into structured meaning.

PHI: Personal Health Information.

PIPEDA: Personal Information Protection and Electronic Documents Act (Canada) governing privacy and data handling.

SPIKES Protocol: Six-step protocol for delivering bad news in healthcare.

TTS (Text-to-Speech): Converts the system's text responses into synthetic speech output.

WCAG 2.1: Web Content Accessibility Guidelines 2.1 — standards ensuring digital accessibility for all users.

10. Appendix A – Dialogue System Architecture Diagram

```
User (Trainee)
 ├── Inputs: Text / Audio (via ASR)
 └── Output: System Response
      │
      ▼
Learner-Facing Web UI (React)
 ├── Chat Interface & Feedback View
 └── Secure Login (SSO)
      │
      ▼
FastAPI Backend (REST)
 ├── Authentication and Session Management
 └── HTTPS Communication
      │
      ▼
Natural Language Understanding (NLU)
 ├── Intent / Entity Extraction
 └── Emotion & SPIKES Detection
      │
      ▼
Dialogue Manager (DM)
 ├── State Tracking and Policy Selection
 └── Integration with Case Repository
      │
      ▼
Model Gateway
 ├── Connects to Gemini / GPT / Local Models
 └── Applies Safety Filters
      │
      ▼
Natural Language Generation (NLG)
 └── Generates empathetic SPIKES-aligned responses (optional TTS)
      │
      ▼
Data & Analytics Layer (PostgreSQL + Admin Dashboard)
 └── Stores transcripts, feedback, and analytics
```

Project Development Plan – MedLLM Interface

1. Team Meeting and Communication Plan

Primary Tools	GitHub Projects (issues + kanban), Jira (Agile sprint management and reporting), Discord (daily communication), Google Docs (collaborative editing), and Microsoft Teams (supervisor meetings).
Meeting Cadence	• Weekly team sync: Fridays @ 5 PM EST • Supervisor check-in: Bi-weekly Wednesdays @ 3 PM EST • Ad-hoc technical huddles as needed via Discord
Collaboration Practices	All code changes are merged through GitHub Pull Requests with a 1-person review policy. Design documents and meeting notes are maintained on GitHub. Major tasks are tracked as GitHub Issues linked to corresponding Jira Epics to ensure end-to-end traceability between commits and sprint objectives.
PM Tool Integration	Hybrid workflow using Github and Jira: Github Issues are used for day-to-day development and version control, while Jira manages Epics, Stories, and sprint progress. TA and supervisor will be added as read-only viewers on the Jira board for transparency and bonus tracking.

2. Team Member Roles

Member	Role / Responsibility	Functional Areas (Requirements Mapping)
Tung Ho	Program Manager / Coordinator	Oversees overall project timeline, sprint planning, and deliverables. Organizes team meetings and maintains notes in /meetings. Ensures all documentation (SRS, Plan, Design, V&V) is up to date and submitted on time. Liaison with instructor/TAs, books meetings, and ensures milestones are met. Assigns P_0/P_1/P_2 priorities and tracks progress. Manages Jira & Confluence dashboards for sprint visibility and reporting.

Asher Haroon	Backend Lead (API Integration & Dialogue System)	Designs and implements the FastAPI backend. Integrates model APIs and backend modules for the Dialogue System (NLU + NLG). Ensures proper error handling, logging, and key security. Collaborates with the database lead for data models and system persistence.
Michael Fedotov	Frontend Lead (UI / UX Development)	Develops the React/Tailwind frontend for the MedLLM Interface. Designs an intuitive chat UI, case selection view, and feedback visualization. Maintains responsive design for both desktop and mobile. Collaborates with the Backend Lead for API integration.
Hammad Ur Rehman	ML Engineer & Evaluation Specialist	Builds empathy heuristics (open-question ratio, reassurance counts). Evaluates bias and fairness across LLM outputs. Assists in testing automated feedback modules and fine-tuning evaluation scripts.
Christian Canlas	Machine Learning & Analytics Lead	Develops and tests empathy and feedback models (e.g., open-question ratio, SPIKES coverage). Implements data pipelines using scikit-learn / NLTK for empathy analysis. Defines performance metrics and validates the accuracy of feedback summaries. Provides benchmark and ablation reports.
Aaryan Kandwal	Testing & Quality Assurance Lead (QA/V&V)	Defines unit and integration tests for backend and frontend components. Leads verification & validation (V&V) activities in the Design Doc. Tracks bugs and regressions through GitHub Issues and Jira. Conducts usability testing and documents results.
Samir Matani	Database & Security Lead	Designs and maintains the PostgreSQL schema for cases, sessions, and feedback data. Implements query caching and database indexing. Enforces data privacy/security compliance and assists with deployment configurations—documents ER diagrams and schema in Confluence.

3. Proof of Concept Demonstration Plan

Goal: Demonstrate that a large language model can simulate a realistic patient conversation and generate useful empathy feedback.

Planned Demo Flow:
1. Trainee logs in as “Dr. Lee.”
2. Selects case: Delivering a Difficult Diagnosis.
3. Engages in text-based chat with an LLM-driven patient.
4. Session concludes with an automated feedback summary (e.g., empathy metrics, open-question ratio).

Proof of Concept Deliverables:
- Functional text-based chat loop between frontend and backend.
- Dialogue System prototype integrating NLU, Dialogue Manager, and NLG.
- Feedback report JSON generation.
- Minimal Admin view (session list → transcript + feedback).
- Export one session JSON via Research API (FR-8).

4. Technology Stack

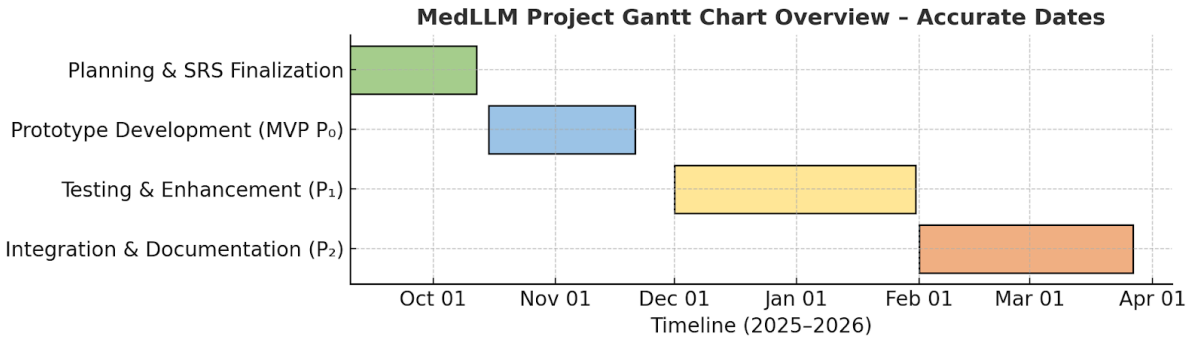
Layer	Technology / Framework	Notes
Frontend	React (TypeScript), Tailwind CSS	Real-time chat updates with WebSocket integration.
Backend / API	Python FastAPI, Pydantic	REST endpoints, Dialogue System logic with replaceable modules, and OpenAPI documentation.
Database	PostgreSQL	Stores user sessions, SPIKES case data, and feedback reports.
Machine Learning / LLM	GPT-4 API (baseline), Hugging Face Transformers	Supports empathy analysis and model testing.
Testing Frameworks	PyTest, React Testing Library	CI tests are automated through GitHub Actions.
Security	HTTPS, Role-Based Access Control	No PHI and no PII are stored. All sessions are anonymized.
GPU Usage	Google Colab / Azure GPU	Used for P_1 and P_2 experiments only.

5. Project Scheduling

Gantt Chart Overview:

Phase	Start Date	End Date	Key Deliverables / Milestones
Planning & SRS Finalization	Sep 10	Oct 12	Submit SRS + Project Plan (v0)
Prototype Development (MVP P_0) - (Delivers FR-1…FR-8)	Oct 15	Nov 21	Implement Web UI, Dialogue System, Admin Logs, Research API export; Proof of Concept demo.
Testing & Enhancement (P_1) - (Delivers FR-10…FR-15)	Dec 1	Jan 31	Add Audio, Model Gateway, Feedback Viz, Analytics Dashboard, Case Authoring, and Bias Mode.
Integration & Documentation (P_2) - (Delivers FR-16…FR-17)”	Feb 1	Mar 27	Integrate multimodal elements, finalize deliverables + video demo

Note: FR-18…FR-20 are future/optional functional requirements.



6. Workflow Plan

6.1 GitHub Usage

Version Control and Branching Strategy

Repository: github.com/AsherHaroon/capstone

Branch Model:

main – Stable release branch containing production-ready code.

dev – Active development branch used for feature integration and testing.

feature/* – Dedicated branches for individual tasks and feature implementations.

Pull Request Policy: All pull requests must include a linked issue reference and receive approval from at least one reviewer before merging.

6.2 Agile Practices

The team follows a Scrum-lite process with 2-week sprints, clear goals, and sprint retrospectives. Sprint boards and user stories are managed in **Jira**, where velocity, sprint progress, and burndown charts are automatically tracked. The **Program Manager** monitors Jira dashboards to review deliverable status and sprint completion metrics.

6.3 Data Management

All synthetic patient data is stored in PostgreSQL with anonymization. Conversation logs and transcripts are versioned via Git LFS.

6.4 Compute Resources

Model experiments run on Google Colab Pro (GPU); only synthetic/redacted data is used outside Canada. Backend runs as a containerized service in a Canadian region (GCP Montréal/Toronto or Azure Canada Central); the database is managed by PostgreSQL in the same region. Static assets may use a CDN, but the origin remains in Canada.

6.5 Requirement Achievement Methods

- Functional Requirements (P_0): Implemented via FastAPI, React, and database integration.
- Non-Functional Requirements: Addressed through automated testing, load testing (Locust), and security enforcement.
- Performance Goals: Response latency ≤ 2.0 seconds average, support 10+ concurrent sessions, and $\geq 90\%$ conversation persistence reliability.
- Fairness/Validity: Bias Probe Consistency $< 10\%$ across demographics; Feedback Reliability $r \geq 0.8$ vs instructor ratings on a pilot set.